1 검증 절차

Django · MySQL 8.4 · ChromaDB · RAG 프로젝트 | Windows PowerShell 및 Docker Compose 기준

목적: 설정·데이터베이스·색인·자동 테스트·AWS 운영 이미지의 정상 동작을 단계적으로 확인한다. 아래 명령은 프로젝트 루트(예: compose.yaml, manage.py 가 있는 폴더)에서 실행한다.

검증 전 준비사항

Docker Desktop 실행, Python 3.12 및 uv 설치, 프로젝트 .env 준비, MySQL 비밀번호와 DJANGO_SECRET_KEY 설정을 확인한다. 실제 비밀키를 문서·로그·Git 에 기록하지 않는다. 개발용 compose.yaml 과 운영용 compose.prod.yaml 은 구분하여 사용한다.

1 단계. 개발용 Docker 설정 확인

Compose 설정을 검증하고 MySQL 등 개발용 서비스를 실행한다.

```
docker compose -f compose.yaml config --quiet
docker compose -f compose.yaml up -d
docker compose -f compose.yaml ps
```

확인 기준: 설정 검증이 오류 없이 끝나고 필요한 서비스 상태가 Up 또는 healthy 로 표시된다.

2 단계. Python 의존성 및 Django 구성 확인

uv.lock 기준으로 개발 의존성을 설치한 후 Django 설정과 마이그레이션 변경 여부를 확인한다.

```
uv sync --dev
uv run python manage.py check
uv run python manage.py makemigrations --check --dry-run
uv run python manage.py showmigrations
```

확인 기준: System check identified no issues 메시지를 확인한다. 마이그레이션 변경이 없으면 makemigrations 검사 명령이 성공한다.

3 단계. 데이터베이스 마이그레이션

개발용 MySQL 이 준비된 상태에서 Django 테이블과 변경 사항을 반영한다.

```
uv run python manage.py migrate
```

확인 기준: Applying ... OK 또는 No migrations to apply 메시지가 표시된다. 운영 데이터베이스에는 백업 후 별도로 적용한다.

4 단계. PDF 및 게시판 RAG 색인

PDF 문서와 게시판을 ChromaDB 에 색인한다. 프로젝트에 index_docs 와 index_boards 관리 명령이 등록되어 있어야 한다.

```
uv run python manage.py index_docs
uv run python manage.py index_boards
```

확인 기준: 두 명령이 오류 없이 종료되고 PDF 및 게시판 검색 결과를 확인할 수 있다. 삭제된 PDF 가 검색되지 않는지도 별도로 시험한다.

5 단계. 자동 테스트 실행

pytest 를 통해 Django 기능과 RAG 관련 테스트를 실행한다. -x 는 첫 실패에서 중단하므로 오류 원인 파악에 유용하다.

```
uv run python -m pytest -vv -s --tb=long -x
```

확인 기준: 전체 테스트가 PASSED 로 종료된다. 실패 시 최초 실패의 traceback 을 확인하고 수정 후 재실행한다.

6 단계. AWS 운영용 Compose 설정 및 이미지 빌드

운영용 Compose 설정을 검증한 뒤 이미지를 빌드한다. Dockerfile 의 collectstatic 은 빌드 시점에 실행되므로 DJANGO_SECRET_KEY 가 필요한 경우 실제 운영 비밀키 대신 빌드 전용 임시 키를 사용하도록 구성해야 한다.

```
docker compose -f compose.prod.yaml config --quiet
docker compose -f compose.prod.yaml build
```

확인 기준: Compose 설정 오류와 Docker 이미지 빌드 오류가 없어야 한다. 빌드 성공만으로 운영 서비스가 정상 작동한다고 판단하지 않는다.

7 단계. 운영 컨테이너 실행 및 로그 확인

운영 환경변수와 HTTPS 프록시 구성이 준비된 경우에만 컨테이너를 실행한다.

```
docker compose -f compose.prod.yaml up -d
docker compose -f compose.prod.yaml ps
docker compose -f compose.prod.yaml logs web --tail=100
```

확인 기준: web·mysql·nginx 서비스가 정상 실행되고 SECRET_KEY 누락, DB 연결 오류, 500 오류가 없어야 한다.

8 단계. 운영 보안 검사

실행 중인 운영 컨테이너에서 Django 보안 검사를 수행한다.

```
docker compose -f compose.prod.yaml exec web .venv/bin/python manage.py check --deploy
```

확인 기준: 보안 경고를 검토하고 HTTPS, ALLOWED_HOSTS, 쿠키, HSTS 설정이 실제 AWS 구성을 반영하는지 확인한다.

수동 통합 검증 체크리스트

□ 회원가입·로그인·로그아웃 정상 동작

□ 게시글 등록·수정·삭제 및 답글 권한 확인

□ 게시글 변경 후 RAG 검색 결과 자동 반영 확인

□ PDF 색인 후 검색, PDF 삭제 후 오래된 검색 결과 제거 확인

□ 정적 파일(CSS/JS) 표시 및 첨부파일 권한 확인

□ HTTPS 접속, HTTP→HTTPS 전환, 세션·CSRF 쿠키 동작 확인

문제 발생 시 점검 순서

1. Docker 설정 오류: docker compose -f compose.prod.yaml config 로 환경변수·볼륨 정의 확인

2. 이미지 빌드 오류: Dockerfile 의 collectstatic 단계와 빌드 전용 SECRET_KEY 설정 확인

3. DB 오류: MySQL healthcheck, MYSQL_HOST=mysql, 포트 3306, 계정 권한 확인

4. RAG 오류: OPENAI_API_KEY, ChromaDB 저장 경로, 컬렉션·메타데이터 및 색인 로그 확인

5. HTTPS 오류: ALB/Nginx 프록시 헤더와 SECURE_SSL_REDIRECT, 인증서 설정 확인

주의: 운영용 compose.prod.yaml 의 실제 서비스명과 이미지 내부 Python 경로가 위 예시와 다르면 해당 파일을 기준으로 명령을 조정해야 한다.